

Работа с датами. Calendar и DatePicker

Структура DateTime

Для работы с датами и временем в .NET предназначена структура **DateTime**. Она представляет дату и время от 00:00:00 1 января 0001 года до 23:59:59 31 декабря 9999 года.

Для создания нового объекта DateTime также можно использовать конструктор. Пустой конструктор создает начальную дату:

```
1 DateTime dateTime = new DateTime();  
2 Console.WriteLine(dateTime); // 01.01.0001 0:00:00
```

Чтобы задать конкретную дату, нужно использовать один из конструкторов, принимающих параметры:

```
DateTime date1 = new DateTime(2015, 7, 20); // год - месяц - день
```

```
Console.WriteLine(date1); // 20.07.2015 0:00:00
```

Установка времени:

```
DateTime date1 = new DateTime(2015, 7, 20, 18, 30, 25); // год - месяц - день - час -  
минута - секунда
```

```
Console.WriteLine(date1); // 20.07.2015 18:30:25
```

Операции с DateTime

Для добавления дат используется ряд методов:

- `Add(TimeSpan value)`: добавляет к дате значение `TimeSpan`
- `AddDays(double value)`: добавляет к текущей дате несколько дней
- `AddHours(double value)`: добавляет к текущей дате несколько часов
- `AddMinutes(double value)`: добавляет к текущей дате несколько минут
- `AddMonths(int value)`: добавляет к текущей дате несколько месяцев
- `AddYears(int value)`: добавляет к текущей дате несколько лет

Например, добавим к некоторой дате 3 часа:

```
1 DateTime date1 = new DateTime(2015, 7, 20, 18, 30, 25); // 20.07.2015 18:30:25
2 Console.WriteLine(date1.AddHours(3)); // 20.07.2015 21:30:25
```

Для вычитания дат используется метод **Subtract(DateTime date)**:

```
1 DateTime date1 = new DateTime(2015, 7, 20, 18, 30, 25); // 20.07.2015 18:30:25
2 DateTime date2 = new DateTime(2015, 7, 20, 15, 30, 25); // 20.07.2015 15:30:25
3 Console.WriteLine(date1.Subtract(date2)); // 03:00:00
```

Здесь даты различаются на три часа, поэтому результатом будет дата "03:00:00".

Метод Subtract не имеет возможностей для отдельного вычитания дней, часов и так далее. Но это и не надо, так как мы можем передавать в метод AddDays() и другие методы добавления отрицательные значения:

```
1 // вычтем три часа
2 DateTime date1 = new DateTime(2015, 7, 20, 18, 30, 25); // 20.07.2015 18:30:25
3 Console.WriteLine(date1.AddHours(-3)); // 20.07.2015 15:30:25
```

Методов форматирования дат

```
DateTime date1 = new DateTime(2015, 7, 20, 18, 30, 25);  
Console.WriteLine(date1.ToLocalTime()); // 20.07.2015 21:30:25  
Console.WriteLine(date1.ToUniversalTime()); // 20.07.2015 15:30:25  
Console.WriteLine(date1.ToLongDateString()); // 20 июля 2015 г.  
Console.WriteLine(date1.ToShortDateString()); // 20.07.2015  
Console.WriteLine(date1.ToLongTimeString()); // 18:30:25  
Console.WriteLine(date1.ToShortTimeString()); // 18:30
```

Calendar и DatePicker.

Calendar представляет собой элемент в виде календаря, тогда как DatePicker - текстовое поле для ввода даты с выпадающим календарем после ввода.

BlackoutDates	Принимает в качестве значения объект CalendarDateRange , задающий с помощью свойств Start и End диапазон дат, которые будут зачеркнуты в календаре.
DisplayDateStart и DisplayDateEnd	Задают соответственно начальную и конечную дату диапазона, который будет отображаться в календаре.
IsTodayHighlighted	Отмечает, будет ли выделена текущая дата
SelectedDate(SelectedDates)	Задаёт выделенную дату (диапазон выделенных дат)
FirstDayOfWeek	Задаёт первый день недели

Calendar

Также Calendar имеет еще два важных свойства: **DisplayMode**(формат отображения дат) и **SelectionMode** (способ выделения).

DisplayMode может принимать одно из следующих значений:

- **Month** (по умолчанию) отображает все дни текущего месяца
- **Decade** отображает все года текущего десятилетия
- **Year** отображает все месяцы текущего года

SelectionMode может принимать одно из следующих значений:

- **SingleDate** (по умолчанию) выделяет только одну дату
- **None** запрещает выделение
- **SingleRange** по нажатию на Ctrl выделяет несколько последовательно идущих дат
- **MultipleRange** по нажатию на Ctrl выделяет несколько не последовательно идущих диапазонов дат


```

<Grid>
  <Calendar x:Name="calendar1" FirstDayOfWeek="Monday"
    SelectedDatesChanged="calendar_SelectedDatesChanged">
    <Calendar.BlackoutDates>
      <CalendarDateRange Start="1/1/2026" End="3/3/2026"></CalendarDateRange>
    </Calendar.BlackoutDates>
  </Calendar>
</Grid>

```

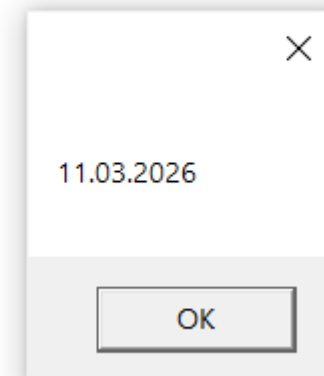
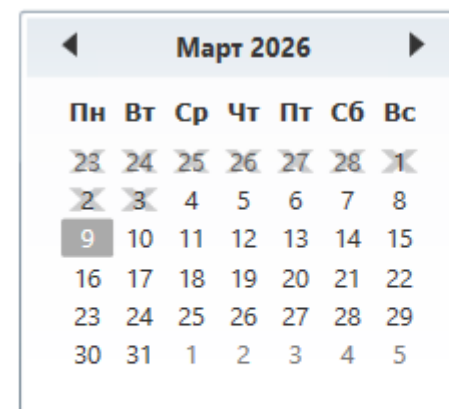
Чтобы использовать в программе выбор даты пользователем, мы можем обработать событие **SelectedDatesChanged** в коде C#:

```

private void calendar_SelectedDatesChanged(object sender, SelectionChangedEventArgs e)
{
    DateTime? selectedDate = calendar1.SelectedDate;

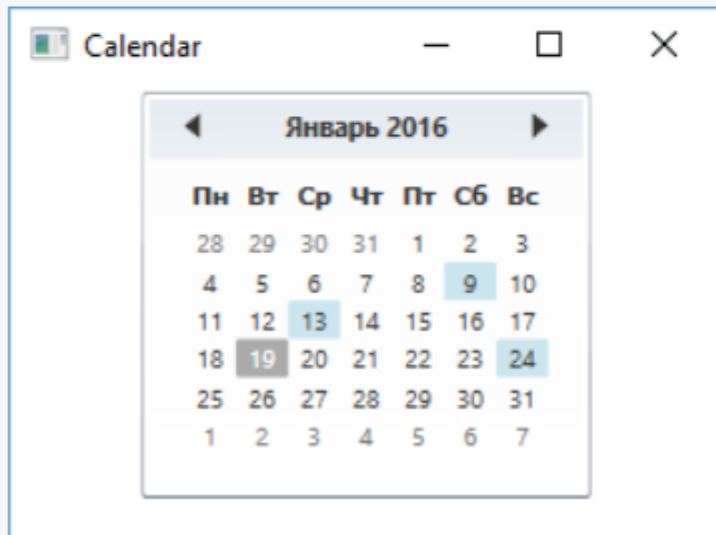
    MessageBox.Show(selectedDate.Value.Date.ToShortDateString());
}

```



Для создания набора выделенных дат нам надо подключить в xaml пространство имен System:

```
1 <Calendar x:Name="calendar1" FirstDayOfWeek="Monday" SelectionMode="MultipleRange"
2     xmlns:sys="clr-namespace:System;assembly=mscorlib">
3     <Calendar.SelectedDates>
4         <sys:DateTime>01/9/2016</sys:DateTime>
5         <sys:DateTime>01/13/2016</sys:DateTime>
6         <sys:DateTime>01/24/2016</sys:DateTime>
7     </Calendar.SelectedDates>
8 </Calendar>
```

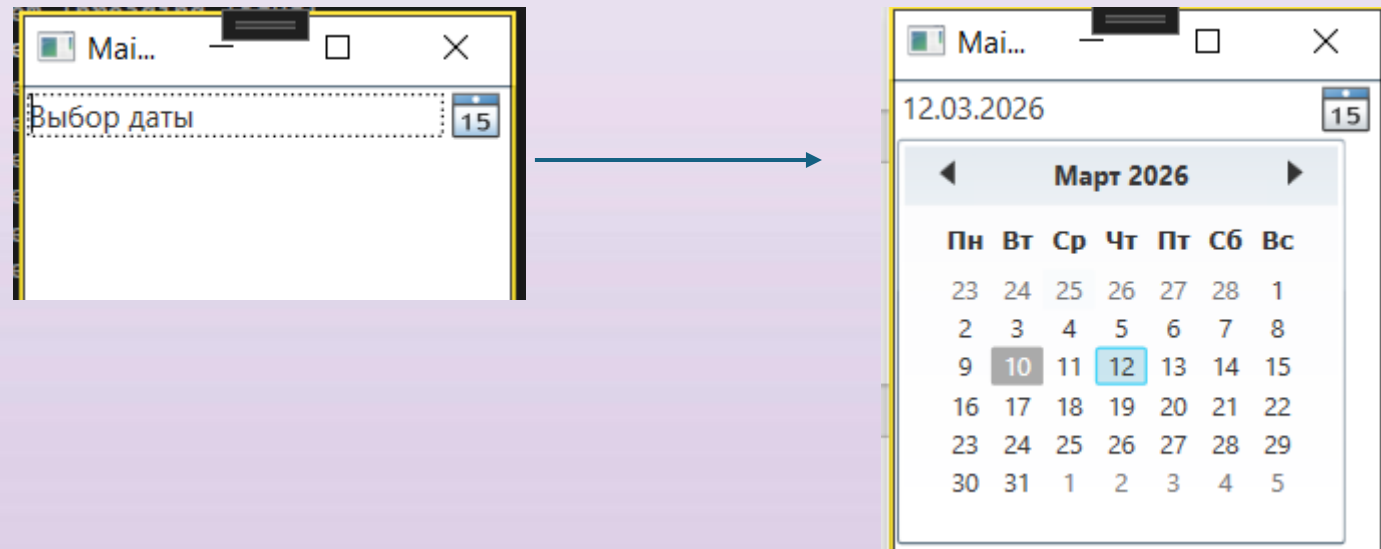


DatePicker

Кроме выше названных свойств элемент DatePicker определяет еще ряд свойств, среди которых **IsDropDownOpen** (показывает, будет ли связанный с элементом всплывающий календарь оставаться открытым после выбора даты) и **SelectedDateFormat** (формат даты, принимает значение Short - формат в виде дд.мм.гггг или мм.дд.гггг и значение Long - с полными названиями месяцев):

```
File MainWindow Height 400 Width 600  
<Grid>  
  <DatePicker Name="datePicker1" IsDropDownOpen="True" SelectedDateFormat="Short" />  
</Grid>
```

И при нажатии на элемент отобразится тот же календарь для дальнейшего выбора даты.



Задание

Пользователь выбирает две даты (одна Calendar и вторая DatePicker).

После выбора двух дат(без кнопки) выводится информация о количестве дней между ними, вывести начало и конец в длинном и коротком формате.

Используйте **BlackoutDates** и **DisplayDateStart**